吸嘟嘟游戏软件
软件设计说明书

版本号：V1.0

1 概述

竖屏休闲收集小游戏，玩家按住屏幕产生一个圆形吸力场，把场景里的灰尘、毛球、碎屑吸进嘟嘟的肚子。吸够目标数过关，章节推进解锁新场景和伙伴。游戏走轻松喜剧路线，嘟嘟是一台有生命的吸尘器，会打喷嚏、会吃饱、会对着脏东西兴奋。

全程纯IAA，不接真实付费，核心进度不依赖广告。引擎用团结，C#开发，逻辑在 `GimmeAll.*` 命名空间下，按 Ads/Audio/Core/Game/Gameplay/Levels/PlayGate/Replay/Social/UI 分模块。

游戏的核心手感在"跟手"和"满足感"。场心跟手用帧率无关 lerp，高帧率和低帧率手感一致，不会因为掉帧出现场心迟滞。捕获瞬间有粒子爆发、音效、HUD 数字跳动三重反馈，玩家每吸到一个东西都有明确的正反馈。目标族重量不同，灰尘一吸就走、碎屑要吸几秒，玩家能直观感受到"重的东西难吸"，不需要看数值表。

章节推进给新场景、新目标族、新伙伴，每章有独立美术和剧情文本。二十多关的内容量，日常关卡轮换保证重复游玩有变化。元系统（签到、图鉴、伙伴、日常、离线、商店）围绕核心玩法循环，不做脱离玩法的独立养成线。

2 系统架构

```
GimmeAll.Core
GimmeAll.Gameplay
GimmeAll.Levels
GimmeAll.Game
GimmeAll.Ads            metrics
GimmeAll.Audio
GimmeAll.UI HUD
```

```
GimmeAll.Social
GimmeAll.Replay
```

核心层不依赖 Unity 的具体表现，Gameplay 层吃 Core 的事件和注册表，Game 层驱动流程。三层边界划清楚后，加新玩法只在对应层动。

3 吸力场 SuctionField

吸力场是整个游戏的手感核心。按住指针产生圆形力场，场心用帧率无关的 lerp 跟手，宁跟手勿迟滞。每帧分两阶段跑：

第一阶段倒序扫注册表，把圈内目标全量登记（供逃脱判定用），同时选出距场心最近的 K 件。K 由 `Config.maxParallelCaptures` 控制，限制并行捕获数是为了不架空逐件计价的时限设计——如果整簇同时攒进度，玩家按住不动就能全收，关卡的时间压力就没了。

第二阶段只对最近 K 件调 `ApplySuction`，进度达阈值当场捕获，发 `ObjectCaptured` 事件。上帧在圈内、本帧出圈的目标发 `ObjectEscaped`，用预分配字典记录上帧状态。

Update 热路径零 GC：不用 LINQ、不用闭包、不拼字符串，字典和候选缓冲都预分配。衰减曲线用 33 段冷采样查表，懒建是因为 Bootstrap 在 AddComponent 之后才赋正式 Config，Awake 期建表会锁死默认曲线。

场视觉有两层细节：按下 0.12 秒弹性长出，不生硬瞬现；平时带 ±3.5% 呼吸脉动，让力场看起来是活的。按压时还有 3 颗星沙在场内螺旋内卷，把拉拽方向可视化，玩家能直观看到吸力往哪走。

```
void Update()
{
//          +    K
//        K  ApplySuction
//               → ObjectEscaped
}
```

吸力场全部手感参数集中在 `SuctionFieldConfig`，一处暴露，调平衡只改这个资产：

参数	值	含义
radius	2.2	力场半径（世界单位）
baseStrength	12	基础吸力
falloffCurve	线性(0,1)→(1,0)	离场心越远力度越弱
followLerp	0.35	每帧跟手闭合比例，1=完全贴手
maxParallelCaptures	3	同时累计捕获进度的目标数上限

maxParallelCaptures 设 3 是反退化设计。没有这个上限时，按住一簇目标的中心，整簇同时攒进度，簇半径上限 1.5 小于场半径 2.2，设计的逐件计价难度（灰尘 0.8 秒/毛发 2 秒/碎屑 3 秒）被 3 到 5

秒站桩清空架空。设成 3 就是"一次吸一小撮"，玩家得移动场心逐簇清理。

```
void ApplySuction(ISuctionable target, float dt)
{
float dist = Vector2.Distance(target.Pos, _fieldPos);
float norm = Mathf.Clamp01(dist / Config.radius);
float strength = Config.baseStrength * Config.falloffCurve.Evaluate(norm);
target.CaptureProgress += strength * dt;
if (target.CaptureProgress >= target.RequiredTime)
Capture(target);
}
```

衰减曲线用 33 段冷采样查表，不是每帧调 AnimationCurve.Evaluate。Evaluate 在热路径里有开销，查表直接索引数组，零计算。表在 Config 更换时懒建并感知更换，不会用到旧曲线。

4 主角与目标族

主角 `ActorDudu` 是一台有生命的吸尘器，有肚子窗口 `BellyWindow` 显示当前吸入量。捕获瞬间播放 `CaptureBurst` 粒子爆发。

目标分四个族，走 `SuctionFamilies2` 统一管理：

族	典型目标	手感
DustTarget	灰尘、尘团	轻，一吸就走
FurTarget	毛球、毛发	中等，需要持续吸
DebrisTarget	碎屑、纸片	较重，吸的时间长
SignatureTargets	章节专属物	特殊触发

每个目标实现 `ISuctionable` 接口，吸力场只认接口不认具体类，加新目标族不用改吸力场代码。目标注册到 `ObjectRegistry`，吸力场从注册表拿全集，逃脱和捕获都走注册表的预分配字典。

目标有迟滞逃逸管线。被吸过但没吸满的目标，松手后不会立刻停在原地，而是带一个残余速度慢慢飘走，模拟灰尘被气流带过的惯性。`SuctionTarget` 里记残余速度，每帧衰减，衰减系数按目标族不同——灰尘衰减快、碎屑衰减慢，重的东西惯性大。迟滞让松手后的画面不僵硬，目标会自然散落。

逃脱判定走预分配字典 `_wasInRange`，键是 `IRecordable`，值是上帧是否在圈内。每帧扫完注册表后，对比上帧和本帧状态，上帧圈内本帧出圈的发 `ObjectEscaped`。字典预分配 256 容量，正常关卡目标数远低于这个值，运行时不扩容。

5 喷嚏系统 SneezeSystem

嘟嘟吸满了会打喷嚏。`SneezeSystem` 取 `SuctionField.FieldPos` 做风暴心，松手后保留最近一次按压场心。喷嚏是一个范围爆发，把附近目标吹飞，是策略元素——玩家可以故意吸满然后在密集区打喷嚏，一次清一片。

喷嚏有冷却，不是随时能放。冷却期间嘟嘟肚子会鼓起来，视觉上提示玩家快满了。

喷嚏的风暴心取 `SuctionField.FieldPos`，松手后保留最近一次按压场心，这样玩家可以先把场心移到目标密集区再松手打喷嚏。风暴是范围爆发，对范围内目标施加一个向外的冲量，同时播放 `CaptureBurst` 的反向粒子。被吹飞的目标如果飞出场景边界，算逃逸，发 `ObjectEscaped`。

```
void TriggerSneeze()
{
Vector2 center = suctionField.FieldPos;
foreach (IRecordable target in registry.All)
{
float dist = Vector2.Distance(target.Pos, center);
if (dist < sneezeRadius)
{
Vector2 dir = (target.Pos - center).normalized;
target.ApplyImpulse(dir * sneezeStrength);
}
}
cooldown = sneezeCooldownSec;
}
```

喷嚏的充能量和捕获数挂钩，每捕获一个目标充一点，满了之后嘟嘟鼻子会亮，提示玩家可以放。玩家也可以选择憋着不放，继续用吸力场逐个吸，喷嚏是策略选项不是强制。

6 关卡系统

关卡规则全部抽成 `LevelRules` 纯函数，可单测：

```
int ResolveGoal(int goalCount, int total) // goal<=0
bool GoalReached(int captured, int goal) //
bool ShouldCountdown(float limitSec) // <=0
bool StepCountdown(ref float remaining, float dt) //
```

`LevelRunner` 驱动单关：加载 `LevelData`，初始化目标生成，跑计时，调 `GoalTracker` 追踪捕获数，达标或超时结算。`LevelTimer` 管倒计时，`timeLimit <= 0` 时秒表照跑但倒计时不跑，做成不限时关。

章节走 `ChapterLore` 和 `ContentCatalog`，每章有独立美术（`ChapterArt`）、剧情文本和目标族组合。`DailyLevelPolicy` 管每日关卡的选取策略，`LevelProgressStore` 持久化关卡进度。

关卡数据用 `LevelData` 序列化，走 JSON，关卡管线、玩法、每日轮换共用一套格式：

```
[Serializable]
public class LevelData
{
public int id;
public string title;
public float timeLimitSec = 60f;
public int goalClearedCount = -1; // -1 =
public List<SpawnEntry> spawns = new List<SpawnEntry>();
}

[Serializable]
public class SpawnEntry
{
public string typeKey; // "dust"/"fur"/"debris"
public float x, y;
public float scale = 1f;
public List<float> props; //
public string spriteKey; //
public int tintIdx; //
public string gearKey; //
}
```

`goalClearedCount = -1` 退化为全清，`ResolveGoal` 里统一处理。`timeLimitSec <= 0` 做成不限时关，秒表照跑但倒计时不跑。加法字段（spriteKey、tintIdx、gearKey）对旧数据向后兼容，缺省为 null 或 0，老关卡不用重新导出。

`LevelRunner` 驱动单关时，先从 `LevelCatalog` 拿 `LevelData`，用 `GameplaySpawnFactory` 按 spawns 列表实例化目标，注册到 `ObjectRegistry`，然后启动 `LevelTimer`。目标被捕获时 `GoalTracker` 计数，达标发 `LevelComplete`，超时发 `LevelFailed`。

7 元系统

元系统全部在 `GimmeAll.Game` 下，各自独立 Flow：

- 签到 CheckInFlow：每日登录签到，连续签到给递增奖励，断签重置
- 图鉴 CodexFlow：收集过的目标族和章节专属物进图鉴，`CodexAccelFlow` 管图鉴加速
- 伙伴 CompanionCoordinator：伙伴有座位（`CompanionSeatFlow`）、升级（`CompanionUpgradeFlow`）、广告加速（`CompanionAdFlow`），伙伴给被动加成
- 日常任务 DailyTaskFlow：每天刷新任务，完成给奖励
- 离线收益 OfflineEarningsFlow：退出后继续产出，回来弹摘要，有上限。进菜单时算一次：离开时长 = now − 上次记录时间，低于 30 分钟不产蛋，超过上限按上限算。产出 = 封顶时长 ×

每小时星沙 / 3600，整数地板。算完立刻刷新记录时间，会话内回菜单复算离开时长约等于 0，天然不重复发。未领的旧蛋在途时跳过新计算，窗口保留不叠加。看激励视频可以翻倍，异点位或无待领时拒付零广播。

```
void ComputeAtBoot(long nowUnix)
{
int seen = MetaStore.OfflineSeenAt;
MetaStore.OfflineSeenAt = (int)nowUnix; //
if (seen <= 0) return; //
if (MetaStore.OfflinePending > 0) return; //
long away = nowUnix - seen;
if (away < MinAwaySeconds) return; //   30
long capped = Math.Min(away, MaxCapHours * 3600);
int sand = (int)(capped * SandPerHour / 3600);
if (sand <= 0) return;
MetaStore.OfflinePending = sand;
}
```

- 伙伴 CompanionCoordinator：伙伴有七族技能，充能靠捕获计数，满了玩家点技能释放。七族分别是：半径扩大（吸力场 BoostRadius）、加时（LevelTimer.Extend）、风暴（SneezeSystem 充满，复用既有喷嚏演出）、星沙直给、星沙乘法窗、护盾（失败时自动复活）、定吸/迟滞。双主战位各自独立充能，关卡开始时解析选席，图鉴管理面选席优先，未选回退名册序。伙伴给被动加成，升级走 `CompanionUpgradeFlow`。
- 商店 StoreFlow：`MetaStore` 管货币和道具，`StoreAdFlow` 管广告兑换

`MetaFlow` 是元系统的总入口，`Bootstrap` 启动时按顺序初始化各 Flow。

签到 `CheckInFlow` 用墙钟 unix 秒算 TodayKey，和日常任务同源口径。连续签到给递增奖励，断签重置。签到面板显示七天循环，已签的打勾，当天的高亮，未来的灰显。补签走广告，`AdConfig` 白名单里配置 checkin placement。

日常任务 `DailyTaskFlow` 每天刷新三个任务，任务类型包括：捕获 N 个目标、完成 N 关、用 N 次喷嚏、收集 N 个毛发等。任务进度实时追踪，完成给星沙奖励。任务定义在 `DailyTaskPolicy`，按日期确定性选取，不用随机API。

图鉴 `CodexFlow` 记录收集过的目标族和章节专属物。每个图鉴条目有首次获得时间、获得总数、稀有度标签。`CodexAccelFlow` 管图鉴加速道具，用了之后一段时间内捕获目标给双倍图鉴进度。图鉴页按族分类，未获得的显示剪影，获得后亮图。

7.5 UI 系统

UI 分 HUD 和弹窗两层。HUD 显示当前关卡目标数、倒计时、星沙数、伙伴充能条，全部实时刷新。弹窗包括签到、图鉴、商店、伙伴、设置、暂停，每个弹窗独立预制体，用栈管理，打开压栈、关闭弹栈，最上层的弹窗拦截输入。

`ICanvasBoard` 接口抽象画布板，背景板 `BackdropBoard` 实现这个接口，每章有独立的背景美术，章节切换时换板。UI 用静态合批，减少 DrawCall。字体用统一的中文 TTF，避免不同平台字体回退导致的排版错乱。

章节宝箱 `ChapterChestFlow` 管章节通关后的宝箱奖励。每章通关后弹宝箱，玩家点开箱动画，给星沙和道具奖励。宝箱有稀有度，普通、稀有、史诗三档，稀有度按章节进度递增，后期章节给更好的奖励。宝箱奖励走确定性规则，不用随机API，同一章同一进度给同样的奖励，避免玩家刷档。

战内道具有两种：沙漏和磁力。沙漏用了加 5 秒，复用 `LevelTimer.Extend`；磁力用了吸力半径 ×1.25，持续 15 秒。道具在 HUD 上显示库存，库存不足时按钮灰显，点了静默拒付。道具通过章节宝箱和商店获得，不接真实付费。

8 技术实现

- 事件总线 `GameEventBus` 解耦模块，事件定义在 `GameEvents`
- 状态机 `GameStateMachine` 管全局状态切换
- 对象池 `ObjectPool` 复用粒子和目标实例，减少 Instantiate/Destroy
- 服务注册 `ServiceRegistry` 管理全局服务的生命周期
- 衰减曲线 33 段查表，热路径零 GC
- 吸力场跟手用帧率无关 lerp，高帧率和低帧率手感一致
- 存档用 JSON 本地存储，自动保存备份

事件总线 `GameEventBus` 用静态委托订阅，按实例去重。`RuntimeInitializeOnLoadMethod(BeforeSceneLoad)` 里 AutoBind，Fast-Enter 模式下静态态须在 SubsystemRegistration 重置，预览会话重开即净面。事件定义在 `GameEvents`，包括 `ObjectCaptured`、`ObjectEscaped`、`LevelStarted`、`LevelComplete`、`LevelFailed`、`GameStateChanged` 等。模块之间只通过事件通信，不直接持有对方引用。

对象池 `ObjectPool` 复用粒子和目标实例。捕获爆发粒子用得最频繁，池容量按关卡最大目标数的两倍预分配，不够时自动扩容但扩容只发生在加载期。目标实例在关卡结束时全部回池，不销毁，下一关直接从池里拿。

8.5 社交与回放

`Social` 模块管分享和截图。`ScreenshotKey` 监听截图快捷键，生成带游戏画面和成绩的分享图。`ShareLandingGate` 管分享落地页的门控，未达成条件的分享入口不显示。

`Replay` 模块录制操作序列。`IRecordable` 接口标记可录制对象，吸力场的按压位置、目标的捕获和逃脱都进录制流。回放时按时间戳重放操作，不重放随机数（游戏本身不用随机API，确定性哈希，所以同一输入序列产出同一结果）。回放用于客服排查和玩家分享高光。

8.6 音频

`Audio` 模块走分层混音。`AmbientLoopLayer` 管环境循环层，每章有独立的环境音，章节切换时交叉淡入淡出。吸力场按压时有持续的吸尘声，音量随场心距目标的距离变化，越近越响。捕获瞬间有短促的"叮"声，不同目标族音高不同，灰尘高、碎屑低，玩家听声音就知道吸到了什么。

喷嚏有专属的爆发音效，带低频冲击。音频分通道调音量，BGM 和音效独立，玩家可以分别关。

8.7 组合根与启动

`Bootstrap` 是全项目唯一的组合根，唯一允许跨模块引用的胶水层。`RuntimeInitializeOnLoadMethod(AfterSceneLoad)` 时启动，先建基础设施（正交相机、吸力场、关卡会话件），再接服务链，最后订阅事件。生产构建中 Bootstrap 就是入口，运行时全自装配，零场景依赖。

视口恒定 11.25×15（3:4），orthographicSize = 7.5。大场景 20.5 世界单位由 CameraRig 巡游，不扩视口。复活奖励 = 续 10 秒。Bootstrap 订阅的事件包括：关卡开始请求、继续请求、每日关卡请求、复活请求、回菜单请求、道具使用请求、关卡开始。每个事件有独立的处理方法，职责单一。

8.8 相机系统

`CameraRig` 在 LateUpdate 里以帧率无关 lerp 跟随吸力场场心（FollowActor 模式下跟随演员位置），followLerp = 0.16，略缓于演员的 0.18，镜头带自重感。

目标位置先钳进"场景包围盒各收缩半个视口"的巡游窗，镜头永不露出场景外。某轴窗宽为负（场景比视口小）时，该轴钳回场景中心，另一轴照常巡游。铁律是禁改 orthographicSize 和 aspect，只动 x/y，z 原值保留。相机、吸力场、背板任何一个缺席就静默留原地，菜单和降级装配不瘫痪。

```
void LateUpdate()
{
Rect scene = backdrop.SceneBounds;
float halfW = cam.orthographicSize * cam.aspect;
float halfH = cam.orthographicSize;
float minX = scene.xMin + halfW, maxX = scene.xMax - halfW;
float tx = minX > maxX ? scene.center.x : Mathf.Clamp(target.x, minX, maxX);
// ty
cam.transform.position = Vector3.Lerp(cam.transform.position,
new Vector3(tx, ty, cam.transform.position.z), followLerp);
}
```

8.9 性能预算

- 吸力场 Update 零 GC，预分配字典和缓冲
- 衰减曲线 33 段查表，不调 AnimationCurve.Evaluate
- 对象池复用粒子和目标，关卡内不 Instantiate/Destroy
- 静态合批 UI，减少 DrawCall
- 目标数上限 256，超过不注册
- 包体控制在 30MB 内
- 微信小游戏首包控制在 12MB 内，超出部分走分包
- 60fps 目标，中低端机 30fps 可玩，帧率只影响画面不影响数值

9 广告与变现

纯 IAA，广告位：离线收益翻倍、伙伴加速、商店兑换、签到补签、复活等，全部可选，不看广告能通关。广告走 `AdConfig` 白名单管理 placement，每个广告位有独立的频次门控 `AdFrequencyGate`，先查类别门控、再拿广告结果、最后消耗本地次数。广告预加载，失败给跳过选项不卡流程，状态不变可重试。`AdMetrics` 记录展示、完成、失败数据，`WeChatAnalyticsService` 上报微信小游戏分析。

广告位的白名单机制保证不会出现未配置的 placement 误触发。每个 Flow 调广告时传 placement 字符串，`AdConfig` 校验是否在白名单内，不在直接拒付零广播。这样加新广告位必须显式配置，不会因为代码拼写错误意外弹广告。

10 容错与性能

- 存档解析失败用备份或全新开局，不丢主流程
- 资源缺失用 `PlaceholderSpriteFactory` 生成占位，不白屏
- 全局异常捕获，广告失败不卡死
- 对象池复用粒子和目标，减少 GC 抖动
- 吸力场热路径零 GC，预分配字典和缓冲
- 纯函数规则层可离线单测，数值表冻结后可回归
- 静态合批、按需加载，包体控制在 30MB 内

吸力场的候选缓冲和逃脱字典都预分配 256 容量，正常关卡目标数远低于这个值，运行时不扩容。衰减查表在 Config 更换时自动重建，不会用到旧曲线。

存档写入用先落临时文件再原子替换的方式，防止游戏在保存瞬间被杀留下损坏文件。读档时逐面校验，某一面解析失败只回退那一面，不拖垮整份进度。首跑零产蛋、未领旧蛋不叠加，这些幂等

设计保证离线收益不会因为会话切换重复发放。

广告失败时状态不变，可重试，不卡流程。广告位白名单校验，未配置的 placement 拒付零广播，不会因为代码拼写错误意外弹广告。异点位调用（比如在离线收益界面调复活广告）拒付零广播，supply 同律，各归其位零串扰。

关卡数据用加法字段向后兼容，旧关卡缺省新字段为 null 或 0，不用重新导出。`LevelJson` 用 `JsonUtility` 序列化，空 JSON 回退到 `{}`，不会因为文件损坏抛异常。

Fast-Enter 模式下静态态须在 SubsystemRegistration 重置，预览会话重开即净面。事件总线的静态委托按实例去重，AutoBind 幂等，编辑器下和各车道自测台共存不冲突。

10.5 设计取舍

吸力场限制同时捕获数为 3，而不是全圈并行，是为了保留逐件计价的时间压力。如果全圈并行，玩家按住一簇目标的中心就能站桩清空，关卡的限时设计就被架空了。3 件并行让玩家必须移动场心逐簇清理，操作有存在感。

喷嚏作为策略选项而不是强制机制，是因为强制放喷嚏会打断玩家的吸尘节奏。玩家可以选择憋着不放、逐个精准吸，也可以吸满了在密集区放喷嚏一次清一片，两种玩法都成立。

离线收益设 30 分钟门槛和上限，是为了避免玩家长时间不登录后回来一次性拿太多，破坏游戏节奏。门槛保证短时间离开不产蛋，上限保证长时间离开也不会一夜暴富。

元系统全部围绕核心玩法循环，不做脱离玩法的独立养成线。伙伴的充能靠捕获，升级给被动加成，日常任务的目标都是"捕获 N 个""完成 N 关"这种核心玩法行为，签到和离线收益给的星沙也是用来在商店买核心玩法相关的道具。

11 版本信息

字段	内容
软件全称	吸嘟嘟游戏软件
软件简称	吸嘟嘟
版本号	V1.0
分类号	30200-0000
开发完成日期	2026年09月30日
开发方式	独立开发
权利取得方式	原始取得
权利范围	全部权利
编程语言	C#
源程序量	23000行
硬件环境	ARM四核及以上移动处理器，2GB以上内存，150MB以上存储空间，支持触控

软件环境	Android 8.0及以上/iOS 12.0及以上，微信小游戏平台，团结引擎